

Binding & Storage Allocation

Scope determines where a name can be referenced, and local referencing determines how a variable's storage behaves once a function is called. Binding ties these two ideas together: it is the process of associating an attribute — such as a type, a value, or a memory address — with a name.

Understanding binding and storage allocation explains why static locals persist, why stack variables vanish after a function returns, and why pointers can reach across function boundaries.

1. What is Binding?

A binding is an association between an entity, such as a variable name, and a property of that entity, such as its type, its value, or the memory location it occupies. The moment at which this association is made is called the **binding time**. Binding time has a major influence on language flexibility, performance, and error-checking: bindings made earlier (e.g. at compile time) tend to be faster to use but less flexible, while bindings made later (e.g. at run time) are more flexible but carry runtime overhead.

2. Binding Times

Binding Time	Example	Notes
Language design time	Meaning of the + operator	Fixed when the language itself is designed
Language implementation time	Range of the int type	Fixed by the compiler/runtime vendor
Compile time	Type of a statically typed variable	Determined when source code is compiled
Link / load time	Address of a global variable	Resolved when modules are combined and loaded
Run time	Value of a local variable	Determined while the program executes

A binding is described as **static** if it is completed before run time and remains fixed for the program's entire execution. A binding is described as **dynamic** if it is established, or can change, during run time. The static-versus-dynamic distinction applies not only to scope (as covered in static and dynamic scope) but also to storage allocation, which is the subject of the next section.

Binding & Storage Allocation

3. Storage Allocation Categories

Every variable needs memory in which to store its value, and the moment that memory is set aside — along with how long it remains reserved — defines the variable's **storage class**. Programming languages generally recognize four categories of storage allocation.

3.1 Static Allocation

Static variables are bound to a single memory cell before program execution begins, and that binding does not change for the lifetime of the program. Global variables and variables declared with the **static** keyword inside a function fall into this category. Because the address is fixed at compile or link time, accessing a static variable is fast and requires no extra runtime bookkeeping.

3.2 Stack-Dynamic Allocation

Stack-dynamic variables are allocated on the call stack when their declaration is encountered during execution, and they are deallocated when the enclosing block or function exits. Ordinary local variables and function parameters in C use stack-dynamic allocation. Because the binding of storage is created and destroyed dynamically, recursive functions can have multiple, independent instances of the same local variable active at once, one per activation record on the stack.

3.3 Explicit Heap-Dynamic Allocation

Explicit heap-dynamic variables are allocated and deallocated at programmer-specified points during execution, using operators or functions such as `malloc()` and `free()` in C, or `new` and `delete` in C++. The storage lives in the heap and is accessed indirectly through a pointer variable.

```
int *p = malloc(sizeof(int)); // heap allocation
*p = 42;
free(p);                      // explicit deallocation
```

3.4 Implicit Heap-Dynamic Allocation

Implicit heap-dynamic allocation occurs whenever a variable is bound to heap storage only when it is assigned a value, with no explicit allocation request from the programmer. Languages such as Python and JavaScript use this approach extensively for objects, lists, and dictionaries. It offers great flexibility, but at the cost of runtime overhead for allocation and garbage collection.

Binding & Storage Allocation

4. Storage Class Comparison

Storage Class	Allocated When	Deallocated When	Typical Region
Static	Before execution begins	Program termination	Static data segment
Stack-dynamic	Block/function entry	Block/function exit	Call stack
Explicit heap-dynamic	Programmer request (malloc)	Programmer request (free)	Heap
Implicit heap-dynamic	On assignment	Garbage collected	Heap

5. How This Connects to Scope and Local Referencing

Scope rules (covered in *Scope & Scope Rules*) describe where a name is visible in the source code, while storage allocation describes how long the underlying memory actually exists. A static local variable, as introduced in *Local Referencing & Parameter Passing*, demonstrates this distinction clearly: its scope is still limited to the function in which it is declared, but its storage class is static, so its lifetime spans the whole program.

Parameter passing also depends directly on storage allocation. When an argument is passed by value, the parameter is a fresh stack-dynamic variable, deallocated as soon as the function returns. When an argument is passed by reference using a pointer, the pointer itself is stack-dynamic, but the address it holds may refer to storage with a completely different lifetime — static, stack-dynamic in an outer frame, or heap-dynamic.

Binding & Storage Allocation

6. Worked Example

The example below combines a static local variable with a pointer parameter, showing binding and storage allocation working together.

```
#include <stdio.h>

// 'history' is static: storage is allocated once, before main() runs
void recordAndDouble(int *value) {
    static int callHistory = 0;
    callHistory++;

    // 'value' is a stack-dynamic pointer parameter
    // it holds the address of a variable owned by the caller
    *value = *value * 2;

    printf("Call #%-d -> new value: %d\n", callHistory, *value);
}

int main() {
    int score = 5;
    recordAndDouble(&score); // score becomes 10
    recordAndDouble(&score); // score becomes 20
    printf("Final score: %d\n", score);
    return 0;
}
```

Output

```
Call #1 -> new value: 10
Call #2 -> new value: 20
Final score: 20
```

Here, `callHistory` keeps counting across both calls because it has static storage, exactly like the `count` variable in the earlier `trackCalls()` example. The pointer value is bound to a different address on each call, but since that address points to the same `score` variable in `main()`, the modification persists in the caller, just as in the `reallyChange()` example.

7. Quick Reference: Binding Properties

- Static binding is fixed before run time; dynamic binding is established, or can change, during execution.
- Storage allocation determines a variable's lifetime; scope rules determine its visibility — related but independent properties.

Binding & Storage Allocation

8. Dangling Pointers and Lifetime Mismatches

A common class of bugs arises when a pointer outlives the storage it refers to. If a function returns the address of a stack-dynamic local variable, that memory is reclaimed the moment the function returns, yet the pointer still appears to be valid. Dereferencing it afterward produces undefined behavior, since the memory may have already been reused by another function call.

```
// INCORRECT: returns the address of a stack-dynamic variable  
int *badFunction() {  
    int localVal = 99;  
    return &localVal; // localVal is destroyed when the function returns  
}
```

To avoid this, a function that must return a usable address should either return the address of a static variable, or allocate the memory explicitly on the heap and hand ownership of that memory to the caller.

9. Conclusion

Binding and storage allocation form the bridge between the compile-time idea of scope and the run-time idea of lifetime. Static binding gives predictable, low-overhead access to data that must persist, while stack-dynamic and heap-dynamic allocation provide the flexibility needed for recursion, dynamic data structures, and parameter passing. Recognizing which storage class a variable belongs to is essential for predicting program behavior, avoiding dangling references, and writing C programs that manage memory correctly.